

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет
"Высшая школа экономики"»

Московский институт электроники и математики им.
А.Н. Тихонова НИУ ВШЭ
Департамент компьютерной инженерии

Курс: Алгоритмизация и программирование

ОТЧЕТ
по лабораторной работе №5

Раздел	Максимальная оценка	Итоговая оценка
Работа программы	1	
Тесты	1	
Правильность алгоритма	3	
Листинг	0,5	
Ответы на вопросы	2	
Дополнительное задание	3	
Итого		

Студент: Солодилов Михаил Анатольевич
Группа: БИВ253
Вариант: 232 (5, 12, 2)
Руководитель:
Оценка:
Дата сдачи:

Оглавление

Задание	2
Листинг программы	3
Распечатка тестов к программе и результатов	7

Задание

1. Создать файл для хранения действительных чисел, вводимых с клавиатуры. Прочитать этот файл и вычислить максимальное среди отрицательных чисел.
2. Создать связанный список для хранения целых чисел. Число записей неизвестно, но можно ввести число, являющееся признаком окончания ввода данных. Для четных номеров вариантов организовать очередь, а для нечетных – стек. Для исходного списка решить следующую задачу: вставить заданное число $A1$ после каждого элемента с нечетным номером.
3. Для полученного списка решить следующую задачу: упорядочить по убыванию методом «пузырька» элементы списка, расположенные после первого положительного элемента.

Листинг программы

```
1 #include <stdio.h>
2 #include <math.h>
3
4 static void read_stdin(const char* save_path);
5 static double find_max_neg(const char* path);
6
7 int main()
8 {
9     puts("Лабораторная работа №5");
10
11     puts("Задание №1");
12
13     read_stdin("data.txt");
14
15     double max = find_max_neg("data.txt");
16
17     if (max == -INFINITY)
18     {
19         puts("В последовательности не было отрицательных чисел");
20     }
21     else
22     {
23         printf("max = %lg\n", max);
24     }
25 }
26
27 static void read_stdin(const char* save_path)
28 {
29     FILE* save = fopen(save_path, "w");
30
31     char buffer[512] = "";
32
33     puts("Введите действительные числа, ввод закончите пустой строкой:");
34
35     fgets(buffer, sizeof(buffer), stdin);
36     while (buffer[1] != '\0')
37     {
38         double n = 0;
39         sscanf(buffer, "%lg", &n);
40         fprintf(save, "%lg\n", n);
41         fgets(buffer, sizeof(buffer), stdin);
42     }
43
44     fclose(save);
45 }
46
47 static double find_max_neg(const char* path)
48 {
49     FILE* f = fopen(path, "r");
50
51     char buffer[512] = "";
52
53     double max = -INFINITY;
54
55     while (fgets(buffer, sizeof(buffer), f))
56     {
57         double n = 0;
58         sscanf(buffer, "%lg", &n);
59
60         if (n < 0 && n > max)
61         {
62             max = n;
63         }
64     }
65
66     fclose(f);
67
68     return max;
69 }
```

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
```

```

4 typedef struct Node
5 {
6     int data;
7     struct Node* next;
8 } Node;
9
10 typedef struct Queue
11 {
12     Node* head;
13     Node* tail;
14 } Queue;
15
16 static void push(Queue* q, int n);
17 static void insert(Node* after, int n);
18 static void queue_dtor(Queue* queue);
19 static void print_queue(const Queue queue);
20 static Queue input_queue(void);
21 static void insert_after_odd(Queue* queue, int a1);
22
23 static Node* find_positive(Queue queue);
24 static void sort(Node* node);
25
26 int main()
27 {
28     puts("Лабораторная работа №5");
29
30     puts("Задание №2");
31
32     Queue q = input_queue();
33     puts("Очередь:");
34     print_queue(q);
35
36     puts("Отсортированный:");
37     sort(q.head);
38     print_queue(q);
39
40     return 0;
41
42     int a1 = 0;
43     puts("Введите A1:");
44     scanf("%d", &a1);
45     insert_after_odd(&q, a1);
46     puts("Очередь:");
47     print_queue(q);
48
49     Node* pos = find_positive(q);
50     if (!pos)
51     {
52         puts("Нет положительных элементов");
53     }
54     else
55     {
56         sort(pos->next);
57         puts("Очередь:");
58         print_queue(q);
59     }
60
61     queue_dtor(&q);
62 }
63
64 static void push(Queue* q, int n)
65 {
66     if (q->tail == NULL)
67     {
68         q->tail = calloc(1, sizeof(Node));
69         q->tail->data = n;
70         q->head = q->tail;
71         return;
72     }
73
74     Node* new_node = calloc(1, sizeof(Node));
75     new_node->data = n;
76
77     q->tail->next = new_node;
78     q->tail = new_node;
79 }

```

```

80
81 static void insert(Node* after, int n)
82 {
83     Node* new_node = calloc(1, sizeof(Node));
84     new_node->data = n;
85     new_node->next = after->next;
86     after->next = new_node;
87 }
88
89 static void queue_dtor(Queue* queue)
90 {
91     Node* cur = queue->head;
92
93     while (cur)
94     {
95         Node* next = cur->next;
96         free(cur);
97         cur = next;
98     }
99
100     *queue = (Queue){};
101 }
102
103 static void print_queue(const Queue queue)
104 {
105     Node* cur = queue.head;
106
107     while (cur)
108     {
109         Node* next = cur->next;
110         printf("%d\n", cur->data);
111         cur = next;
112     }
113 }
114
115 static Queue input_queue(void)
116 {
117     Queue q = {};
118
119     char buffer[512] = "";
120
121     puts("Введите целые числа, ввод закончите пустой строкой:");
122
123     fgets(buffer, sizeof(buffer), stdin);
124     while (buffer[1] != '\0')
125     {
126         int n = 0;
127         sscanf(buffer, "%d", &n);
128         push(&q, n);
129         fgets(buffer, sizeof(buffer), stdin);
130     }
131
132     return q;
133 }
134
135 static void insert_after_odd(Queue* queue, int a1)
136 {
137     size_t n = 1;
138     Node* cur = queue->head;
139
140     while (cur)
141     {
142         Node* next = cur->next;
143         if (n % 2 == 1)
144         {
145             insert(cur, a1);
146         }
147         cur = next;
148         n++;
149     }
150 }
151
152 static Node* find_positive(Queue queue)
153 {
154     Node* cur = queue.head;
155

```

```

156     while (cur)
157     {
158         if (cur->data > 0)
159         {
160             return cur;
161         }
162         cur = cur->next;
163     }
164
165     return NULL;
166 }
167
168 static void sort(Node* node)
169 {
170     Node* n1 = node;
171
172     while (n1)
173     {
174         Node* n2 = n1->next;
175         while (n2)
176         {
177             if (n1->data < n2->data)
178             {
179                 int t = n2->data;
180                 n2->data = n1->data;
181                 n1->data = t;
182             }
183             n2 = n2->next;
184         }
185         n1 = n1->next;
186     }
187 }

```

Распечатка тестов к программе и результатов

№	Исходные данные	Результат
1.1	$S[0 : 1] = \{0, 1\}$	В последовательности не было отрицательных чисел
1.2	$S[0 : 3] = \{5, 3, -4, 1\}$	max = -4
1.3	$S[0 : 4] = \{5, 0, -6, -3, 1\}$	max = -3
2.1	$S[0 : 2] = \{0, 1, 2, 3, 4\}, A_1 = -45$	Очередь: 0 -45 1 2 -45 3 4 -45 Очередь: 0 -45 1 4 3 2 -45 -45
2.2	$S[0 : 1] = \{-5, -3\}, A_1 = -100$	Очередь: -5 -100 -3 Нет положительных элементов